SN Computer Science (2020) 1:362
https://doi.org/10.1007/s42979-020-00380-z

ORIGINAL RESEARCH

# Performance Analysis of WebRTC and SIP-based Audio and Video Communication Systems

T. P. Fowdur[1] · N. Ramkorun[1] · P. K. Chiniah[1]

Received: 15 October 2020 / Accepted: 19 October 2020 / Published online: 2 November 2020
© Springer Nature Singapore Pte Ltd 2020

## Abstract

Video and audio communications have become an integral part of all spheres of life. Two commonly used real-time communication protocols for IP-based video and audio communications are the session initiation protocol (SIP) and web real-time communications (WebRTC). These two protocols have been widely used in softphone and video conferencing applications. The main aim of this paper is to make a comparative analysis of the performance of client server applications for video and audio communications developed by both SIP and WebRTC. The SIP system comprises of a desktop client developed in C#, a mobile client developed in Android studio and a FreeSwitch server. For the WebRTC application, the client was developed in JavaScript and the server in Node.js. The WebRTC application also included a speech translation API for real-time speech translation and employed two different codecs namely VP8 and VP9 via a modification of the Session Description Protocol (SDP) header. Results showed that WebRTC peer-to-peer audio communication provides a much lower quality in terms of SSNR than the SIP audio calls. However, for video communications, WebRTC provides better quality in terms of PSNR than SIP. The performance of the WebRTC application was also evaluated with the different codecs in terms of PSNR as well as the impact of the real-time speech translation delay.

**Keywords** WebRTC · SIP · SDP · QoS · SSNR · PSNR

## Introduction

The internet is widely used for different applications, such as e-shopping, browsing, and communication. The internet has assisted the evolution of communication by providing easy access to real-time communication through third-party applications such as Skype, WhatsApp, and other platforms. Two commonly used frameworks for developing real-time communication applications are the web real-time communication (WebRTC), which is an open source framework and the sessions initiation protocol (SIP). Using WebRTC, current web browsers can exchange video and audio (in a peer-to-peer manner) with other capable browsers. Browser-to-browser communication has proven to be a groundbreaking evolution as it permits web developers to build real-time multimedia applications which operate plugin-free, that is, those applications can work without proprietary plugins like Adobe Flash [1]. WebRTC has been standardized by International communities such as The World Wide Web Consortium (W3C) and The Internet Engineering Task Force (IETF) which are working hand in hand into elaborating JavaScript APIs, HTML tags, and communication protocols [2] to upgrade the quality of the video and audio exchanged between peers in different types of applications. SIP, on the other hand is an application-layer control protocol developed by the Internet Engineering Task Force (IETF). It is the implemented standard for initiating, altering, and suspending an engaging user session which includes multimedia elements for instance voice, video and instant messaging. SIP's main aim is communication among multimedia devices. Through two protocols, SIP makes communication possible: real-time transport protocol (RTP)/real-time transport control protocol (RTCP) and session description protocol (SDP) [3]. Given the impact and widespread adoption of SIP and WebRTC in real time

✉ T. P. Fowdur
p.fowdur@uom.ac.mu

N. Ramkorun
nilesh.ramkorun@umail.uom.ac.mu

P. K. Chiniah
pravish.chiniah@umail.uom.ac.mu

[1] Department of Electrical and Electronics Engineering, University of Mauritius, Reduit, Mauritius

communication applications, several researchers have proposed techniques for analyzing and improving their performance. An overview of such research works is given next.

In [4], a voice/video over IP communication system was presented and outlined the dissimilarities and performance between H.323 and SIP (two major VoIP protocols) taking into consideration the packet delay variation, end to end delay, packet loss and jitter. The authors in [5] employed numerous methods to diagnose DoS attacks and distinguish between the frameworks suggested to recognize anomalies, and then inform end users about potential intrusions in a SIP network. In [6], the authors brought up some SIP multimedia system conferencing architectures with their benefits, disadvantages, and performance. The entire architecture of the practical system was introduced and developed. The control mechanism based on SIP for dispatching the scalability was devised in detail and the system's overall performance was assessed and reviewed. SIP was found to provide an acceptable multimedia conferencing platform that supports advanced scenarios and services. To discuss QoS of VoIP calls in Mobile Ad Hoc Networks (MANETs), network bandwidth and suitable routing protocols must be taken into consideration along with its advantages and disadvantages to obtain reasonable delays for various stages like call registration, establishment, retransmission, and termination as pointed out in [7]. The authors performed in [7] performed an analysis of three routing protocols and the SIP signaling and VoIP quality of service were scrutinized on numerous active calls. Other interesting works on SIP can be found in [8] and [9].

Over the past few years, various researchers have tried to improve the quality of video streaming through WebRTC to enhance the overall experience quality. In [10], the authors proposed the implementation of a WebRTC hybrid signaling mechanism using Socket.io and Firefox which would offer multi-party video conferencing by making use of different types of topologies such as mesh, simplex, and star. The goal of the authors in [11] was to explore how the various video resolutions have influenced quality of experience (QoE) in multi-party video conferencing. They conducted a subjective analysis of audiovisual conversations carried out using mobile devices. They also investigated the impact bandwidth limitations had on QoE for various video resolutions. In [12], Maruschke et al. analyzed the impact clients using WebRTC had on QoS which were connected together using an IP multimedia subsystem (IMS)-based IP television (IPTV) infrastructure. They suggested an approach where a WebRTC user can take advantage of incorporated quality for real-time IPTV service inside a 4G mobile network. An experimental contact center WebRTC application in [13] was developed in which the authors investigated on the different WebRTC functions to be able to set up multiple communication channels allowing tasks like calls and files sharing to be done through WebRTC. A performance analysis of WebRTC and session initiation protocol (SIP) [14] for video conferencing was conducted in [15] and it wasfound that WebRTC provided a better call quality compared to SIP under the same network conditions. Other related works on the performance of WebRTC can be found in [17–22].

The contribution of this paper is twofold. First a comparative analysis of both SIP and WebRTC is made for both video and audio communications in a laboratory test bed. For audio transmission, the SSNR has been used as a metric and for video transmission, PSNR has been used. Second, the performance of WebRTC for video conferencing has been further analysed by considering the impact on the PSNR when two different codecs namely VP8 and VP9 are used with and without optimizations. The impact on the transmission delay when a real-time translation API is embedded into the WebRTC application is also assessed. A description of how the SIP desktop and mobile clients are developed as well as the use of the FreeSwitch server is also presented. For the WebRTC application, a detailed description of the client application, the SDP codec optimizations and incorporation of the real-time speech translation API are given along with the details of the Node.js server.

The organization of this paper is as follows. Section 2 provides a detailed overview of WeRTC and SIP based communication systems. Section 3 describes the implementation of the WebRTC and SIP clients and servers. Section 4 presents the results obtained as well as the experimental set-ups used for the analysis. Section 5 concludes the paper with some recommendations for future works.

## Overview of WebRTC and SIP-based Communication Systems

In this section, an overview of WebRTC and SIP-based communication architectures and protocols is provided.

### WebRTC Architecture, APIs, and Protocols

Nowadays, real-time communications can be done through web real-time communication (WebRTC), which is an open source framework. Using WebRTC, current web browsers can exchange video and audio (peer-to-peer manner) with other capable browsers. Figure 1 [23] shows the internal architecture of WebRTC.

Browser-to-browser communication has proven to be a groundbreaking evolution as it permits web developers to build real-time multimedia applications which operate plugin-free, that is, those applications can work without proprietary plugins like Adobe Flash [2]. WebRTC has been standardized by International communities such as The World Wide Web Consortium (W3C) and The Internet

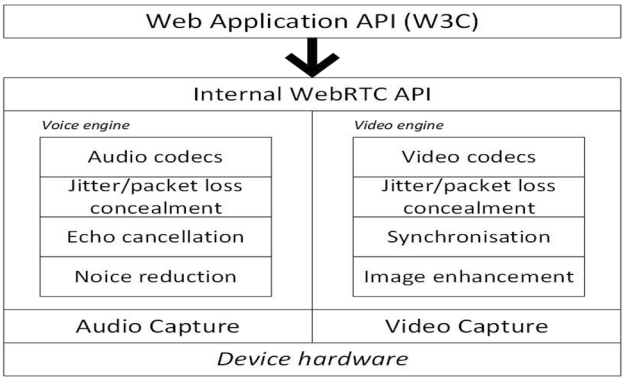



**Fig. 1** WebRTC internal architecture

Engineering Task Force (IETF) which are working hand in hand into elaborating JavaScript APIs, HTML tags, and communication protocols [3] to upgrade the quality of the video and audio exchanged between peers in different types of applications. Over the past years, different protocols, standards, and APIs have been implemented into the underlying WebRTC architecture to improve the Quality of Service (QoS).

WebRTC implements three APIs:

- MediaStream
- RTCPeerConnection
- RTCDataChannel

### MediaStream

The MediaStream API allows users to access media streams from the local entry terminals for instance cameras and microphones [24].

It gives entry to a stream entity containing the video and audio streams.

MediaStream API has three main functionalities:

- It enables entry to a stream object that constitutes of the video and audio streams.
- It manages input devices when a user has several cameras or microphones on his/her computer.
- Delivers a security level prompting user every time he/she wants to fetch the stream.

### RTCPeerConnection

The RTCPeerConnection API is the core link between each WebRTC peer-to-peer connection [25].

A peerconnection object is being created as follows:

var peerconnection = RTCPeerConnection (configuration).

The caller's flow after the peerconnection object is as follows:

1. Register the onicecandidate handler.
2. Register the onaddstream handler.
3. Register the message handler.
4. Use getUserMedia to access the local camera.
5. The JSEP offer/answer process.

The callee's flow which responds with an answer to the offer of the caller is as follows:

1. Register the onicecandidate handler.
2. Register the onaddstream handler.
3. Register the message handler.
4. Use getUserMedia to access the local camera.
5. The JSEP offer/answer process.

Figure 2 illustrates the WebRTC peer-to-peer video call process.

The steps of Fig. 2 above are depicted below:

1. The initiator makes a search for getUserMedia() to access the MediaStream audio/video and connects it to a new RTCPeerConnection.
2. The RTCPeerConnection is used to generate the Session Description Protocol (SDP) offer.
3. The SDP offer generated is set as the localDescription of the peerconnection and transmitted via Node.js (signaling server) to the receiver.
4. Once the receiver receives the offer, an RTCPeerConnection is generated using its own MediaStream and the received offer is set as its remoteDescription.
5. The receiver uses its RTCPeerConnection to generate a response(Session Description Protocol), then adds the generated response as its localDescription and delivers this answer through the signaling server to the initiator.
6. The initiator gets the answer and sets it as their remoteDescription.
7. Video peer-to-peer call begins, and stream sharing occurs to view each peer's local and remote data.

Now that both peers have shared each other's SDPs and arranged the configuration and form of streams to be used, it is then possible to create a direct connection between the two peers.

The session description protocol (SDP) is a protocol used to define the communication parameters for media streaming. Session announcement, session invitation, and parameter negotiation are provided by SDP. SDP by itself does not provide any media transfer but is used between peers to share associated properties often called a session profile. The session profile regroups the transport

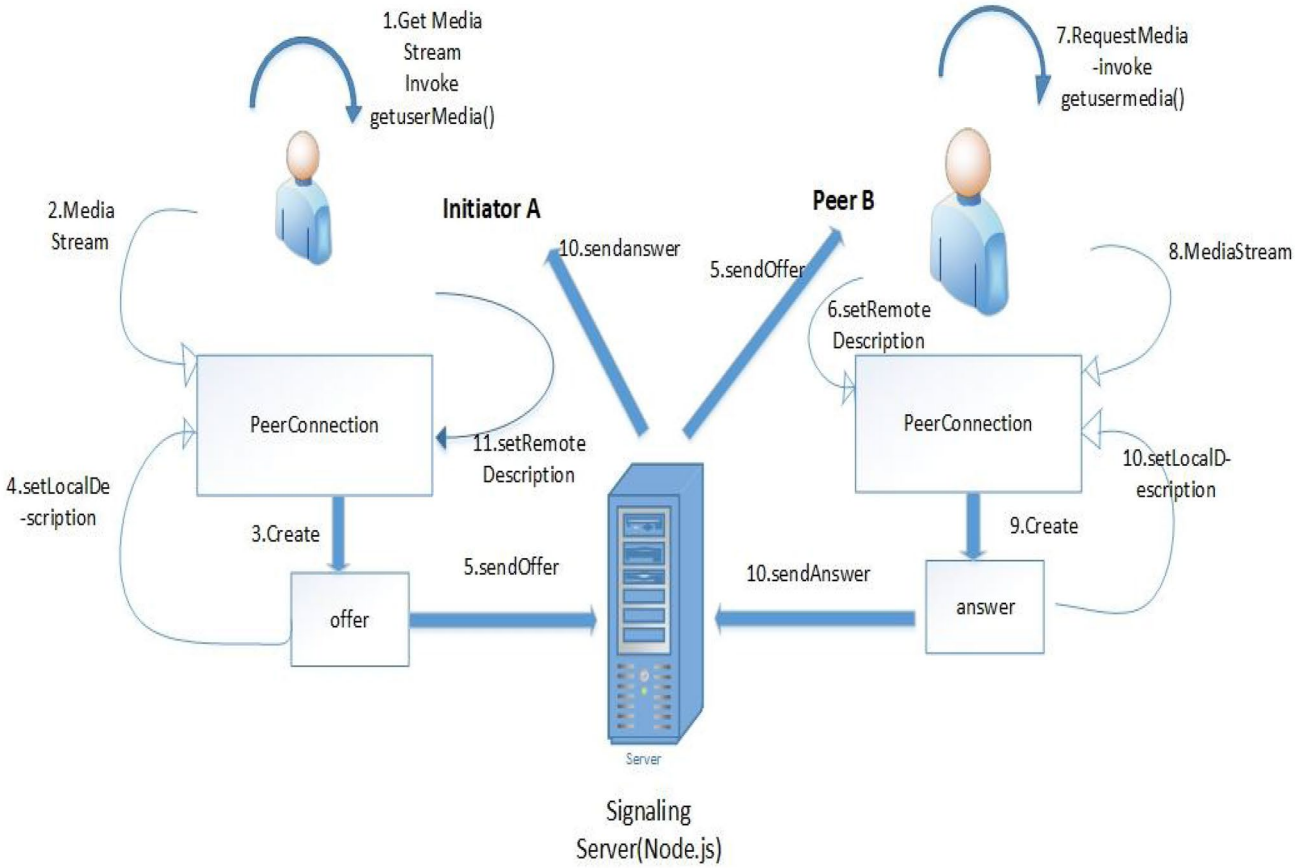



**Fig. 2** WebRTC negotiation process

information, Interactive Connectivity Establishment [ICE] information and all media configuration parameters to establish the media path. Before peers can establish a direct connection, they must first exchange SDP data using the signaling channel. The following is an example of an SDP offer (Figs. 3, 4):

Here, we can see that this is a video and audio session, and multiple codecs are offered.

The following is an example of an SDP answer:

### RTCDataChannel

RTCDataChannel allows the exchange of data between two WebRTC peers with tailored distribution for the transport delivery characteristics. The WebRTC application which uses this channel is able to configure it to provide a reliable delivery of messages is a bidirectional channel, which is part of the PeerConnection API [26].

Other important components of WebRTC include Network Address Translation (NAT) [27], STUN AND TURN

**Fig. 3** WebRTC negotiation process

```
v=0
o=alice 2890844526 2890844526 IN IP4host.atlanta.example.com
s=
c=IN IP4host.atlanta.example.com
t=0 0
m=audio 49170 RTP/AVP 0 8 97
a=rtpmap:0PCMU/8000
a=rtpmap:8PCMA/8000
a=rtpmap:97iLBC/8000
m=video 51372 RTP/AVP 31 32
a=rtpmap:31H261/90000
a=rtpmap:32MPV/90000
```

3 Audio codecs

3 Video codecs

**Fig. 4** SDP Answer

```
v=0
o=bob 2808844564 2808844564 IN IP4host.biloxi.example.com
s=
c=IN IP4host.biloxi.example.com
t=0 0
m=audio 49174 RTP/AVP 0
a=rtpmap:0PCMU/8000
m=video 49170 RTP/AVP 32
a=rtpmap:32MPV/90000
```

Servers [28], Interactive Connectivity Establishment (ICE) [29] and a Signaling server.

## SIP Architecture, APIs, and Protocols

Session initiation protocol as established in IETF RFC 3261 is a real time signaling protocol utilized for creating, modifying, and terminating an engaging user session which includes multimedia elements for instance voice, video and instant messaging. SIP can be used to create a complete multimedia architecture with other IETF protocols**.**

The protocols are RTP/RTCP for real-time data transmission and QoS info. SDP to define multimedia sessions, and MSRP to hold data messages in a chat session. SIP can also be used in combination with other protocols to provide the users with full services.

SIP combines features of two main Internet protocols which are HTTP and SMTP. It can be used to provide multimedia services between:

- call servers;
- application server (AS);
- media resource function (MRF).

Figure 5 depicts the four-layer internet multimedia protocol stack. The application layer is the top layer. It comprises signaling protocols such as SIP and real-time transport protocol (RTP) which is a media transport protocol.

Figure 5 contains H.323 which was initially established by the International Telecommunication Union (ITU) as an equivalent signaling protocol to SIP.

Session overview protocol (SDP) is seen in the protocol stack above, as it is held in a body of SIP messages. SDP allows identification of the media (audio or video) participating in the multimedia session set up with SIP.

The interaction with both TCP, UDP and possibly SCTP (stream control transmission protocol) is shown since SIP can work with any transport protocol [30].

## SIP Entities

A SIP architecture consists of the following entities:

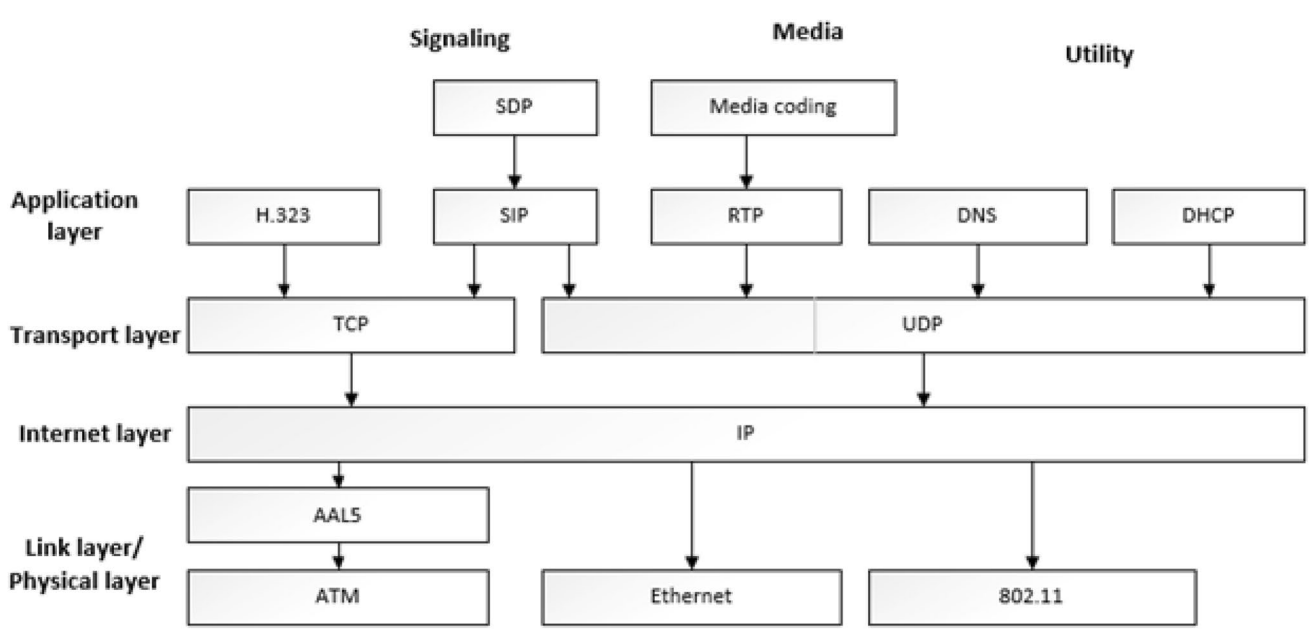



**Fig. 5** Four-layer internet multimedia protocol stack

A user agent consists of:

- user agent client (UAC);
- user agent server (UAS).

A UAC is an application that initiates a SIP request.

A UAS is an application that contacts a user while getting a SIP request and acts on the user's name. The request is accepted, rejected, or redirected by the response.

A proxy server functions both as a server and a client. It responds to requests from clients, modifies them and passes the requests to other servers.

A redirect server handles client requests and renders zero addresses or more. It does not trigger requests or accept calls like a UAS which is different to a proxy server.

The job of the registration server is to accept REGISTER requests. Typically, a registration server is synchronized with a proxy or redirect server [30] (Fig. 6).

## Implementation of the Proposed WebRTC and SIP Communication Systems

In this section a description of both the WebRTC and SIP client and server systems implementations used in this work is given.

### WebRTC Client Server Implementation

The complete system architecture is shown in Fig. 7.

The system consists of the application side which is written in JavaScript and the signaling server which is implemented using Node.js. The application basically performs video conferencing but includes two features namely real-time translation and the use of FEC codecs to improve the performance.

After the server is launched, the server starts listening on port 3000 on which the Home page can then be accessed using the initiator's IP address as shown in Fig. 8. JavaScript and HTML codes have been used to implement most of the functions.

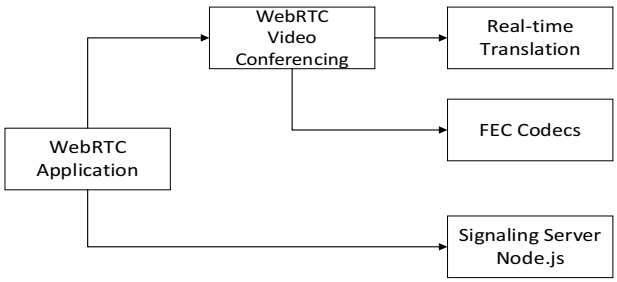



**Fig. 7** WebRTC system architecture

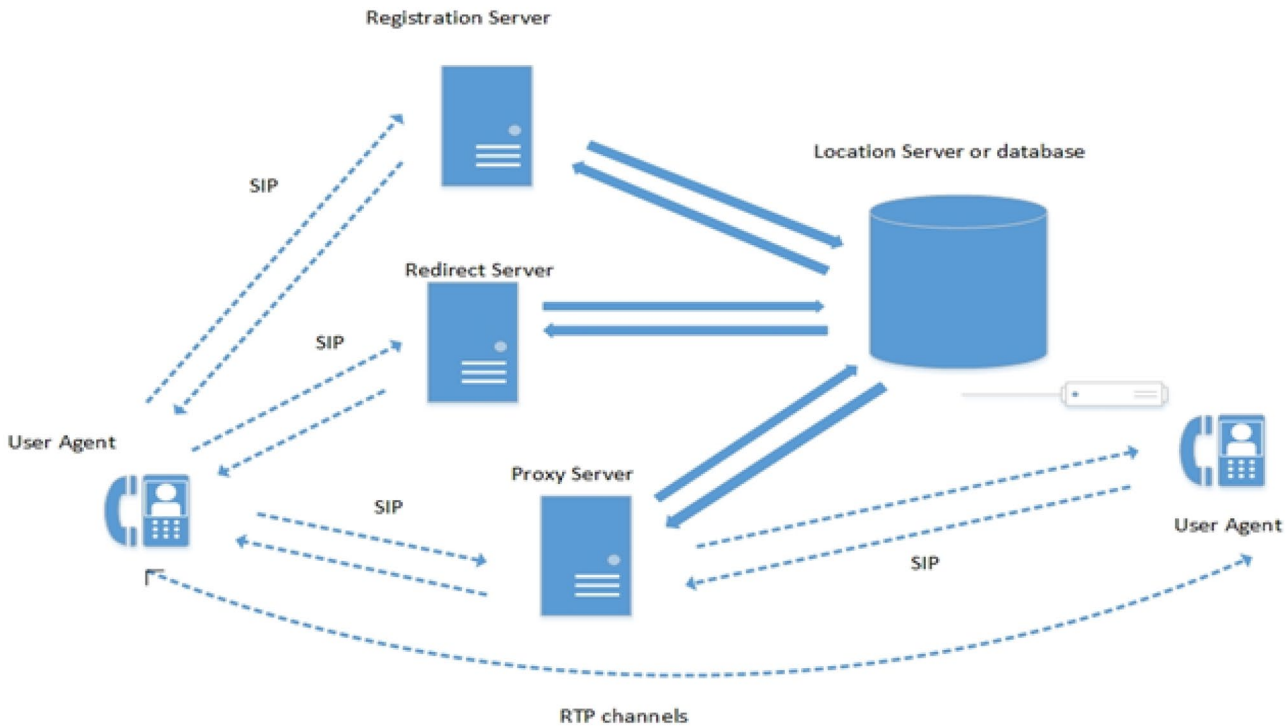
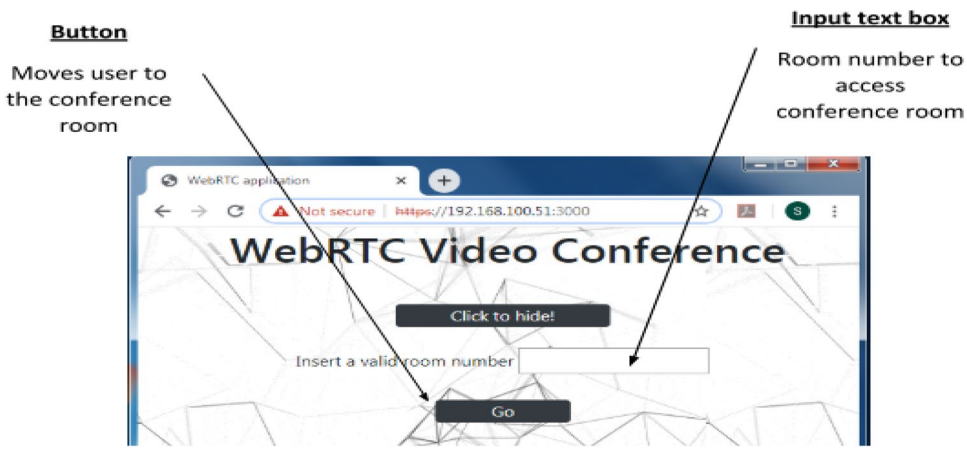



**Fig. 6** SIP Architecture

**Fig. 8** Home Page




The program structure of video conferencing application is shown in Fig. 9.

The video conference room supports the local and remote videos (peers), a real-time voice and text translation option as well the use of FEC. A summary of the main functions shown in Fig. 9 is given Table 1.

### Changing SDP Parameters

The chooseCodec(), setFlexFec(), setGroup(), and setMultipath() functions are used to amend both the (SDP) offer and (SDP) answer. The (SDP) offer is amended because the offer is a way of broadcasting to other peers what the sender can provide. Then, on receiving the offer on the receiver side, the receiver will decide whether it can work together with what the sender is providing.

The chooseCodec() function allows the users to change the video codec in the video conference. Users can choose between the VP8 and VP9 codec. Normally, the (SDP) offer includes all the supported video codecs below the 'm-line'. Figure 10 shows the (SDP) offer video media descriptions.

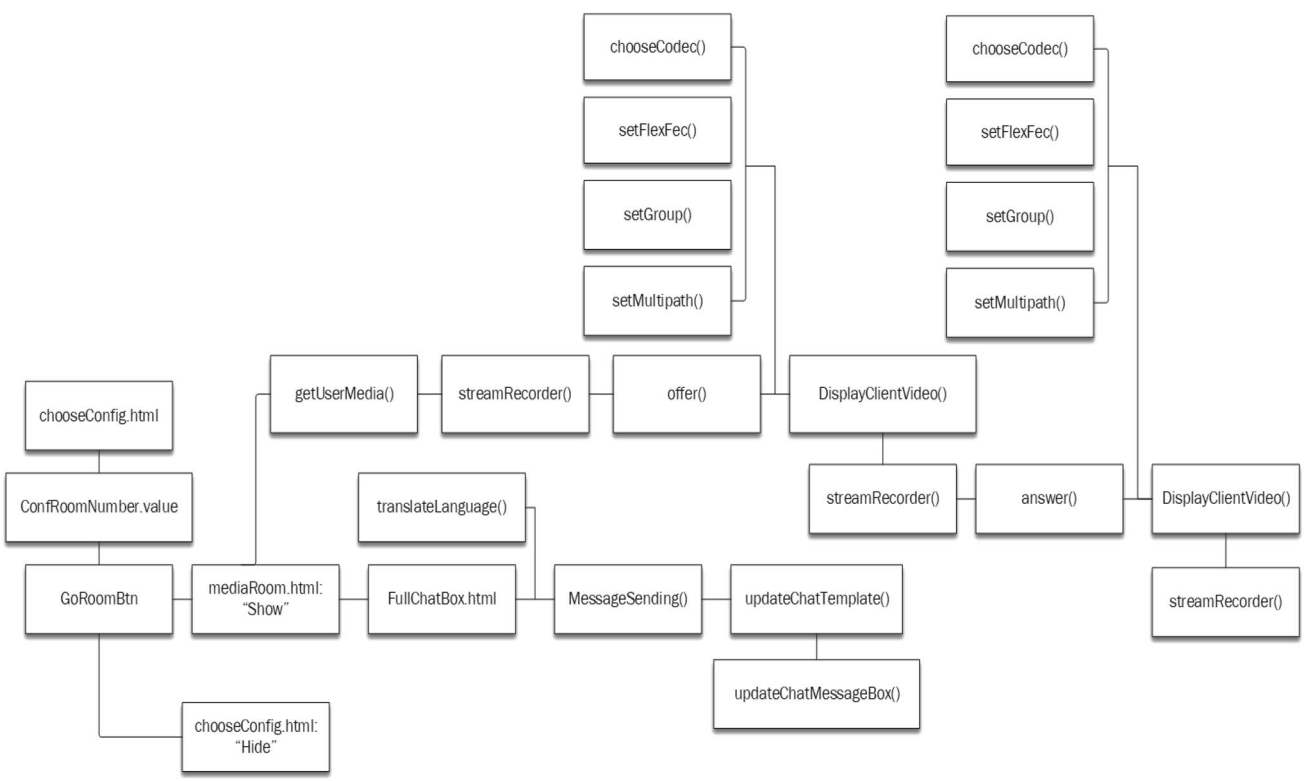



**Fig. 9** Video conferencing application code structure

**Table 1** Description of the main functions

| Function | Description |
|---|---|
| chooseConfig.html | This is the HTML home page shown in Fig. 8 |
| ConfRoomNumber.value | Refers to the room number value for the video conferencing section |
| GoRoomBtn | Button that user clicks to access the video conferencing room and after button is clicked, the 'chooseConfig.html' page is hidden and the 'mediaRoom.html' page is shown, which is the video conferencing room |
| mediaRoom.html | The page that displays the Video conferencing room |
| offer() | Creates a new RTCPeerConnection and uses the generated RTCPeerConnection to create an (SDP) offer |
| DisplayClientVideo() | Displays the other peer's video |
| answer() | Creates a new RTCPeerConnection and uses the generated RTCPeerConnection to create an (SDP) answer |
| FullChatBox.html | This is the message chat box whereby all chat messages are viewed |
| translateLanguage() | Found inside the updateChatMessageBox() function and is responsible for dealing with the translation part |
| MessageSending() | Responsible for sending chat messages across the peers |
| updateChatTemplate() | Provides an HTML template for the message to be displayed on the chat box |
| updateChatMessageBox() | Responsible for updating the chat box for every text message and files sent or received in real-time |
| chooseCodec() | Allows the users to change the video codec in the video conference. Users can choose between the VP8 and VP9 codec |
| setFlexFec() | Changes the FEC implementation from ULPFEC to FlexFec, which offers new RTP payload formats for the FEC |
| setGroup() | Changes the grouping semantic of the SSRC in the SDP |
| setMultipath() | Introduces the multipath parameter in the (SDP) offer and (SDP) answer |

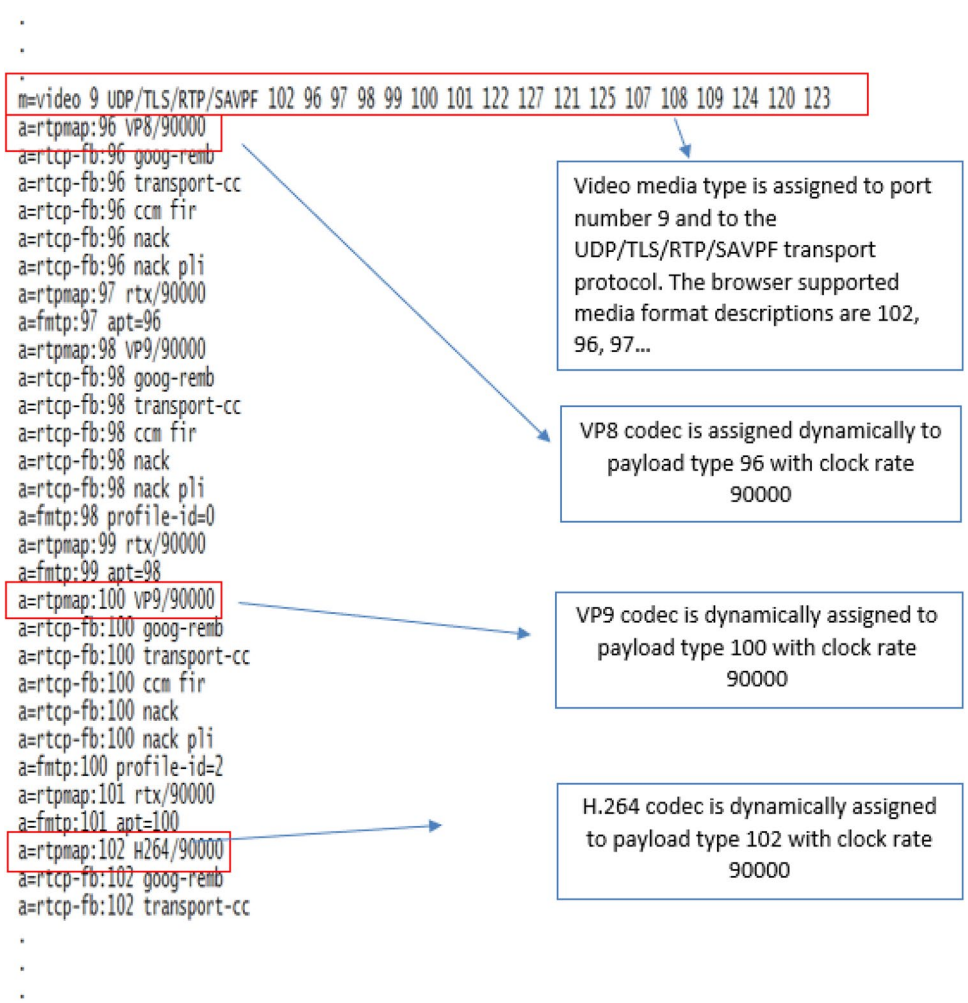



**Fig. 10** Video format descriptions in (SDP) offer

Figure 11 shows the (SDP) answer when the receiver selects the VP8 codec. The media description of VP8 is appended at the start of the media descriptions list shown by the green boxes in the figure.

Figure 12 shows the (SDP) answer when the receiver selects the VP9 codec. The media description of VP9 is appended at the start of the media descriptions list shown by the green boxes in the figure.

The setFlexFec() function changes the FEC implementation from ULPFEC to FlexFEC, which offers a new RTP payload format for the FEC.

The setFlexFec() performs the following steps in the (SDP) offer and (SDP) answer:

1. Split SDP up to 'a = rtpmap:116 ulp/90,000'
2. Replace 'ulp' with 'flexfec'

The application uses the setGroup() function to change the grouping semantic of the SSRC in the SDP. The function performs the following steps in the (SDP) offer and (SDP) answer:

1. Split SDP up to 'a = ssrc-group:FID'
2. Replace 'FID' with 'FEC-FR'

The setMultipath() function introduces the '*a = mprtp*' parameter under the *'a = rtcp-rsize'* line. The function performs the following steps in both the (SDP) offer and (SDP) answer:

1. Split SDP up to 'a = rtcp-rsize'
2. Append 'a = mprtp' just below 'a = rtcp-rsize'

Figure 13 demonstrates when the application has performed all the functions defined above, namely setFlexFec(), setGroup() and setMultipath(). The different parameters that have been changed or appended are highlighted in the figure.

### Real-Time Translation

The Google Cloud Translation API offers a simple interface for the translation of a string of text into any supported language. Google Cloud platform provides different types

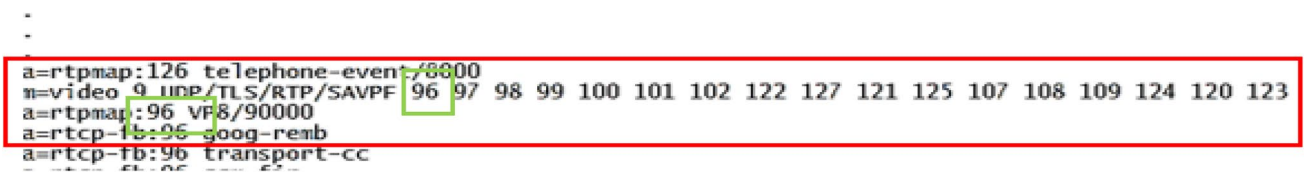



**Fig. 11** VP8 in answer

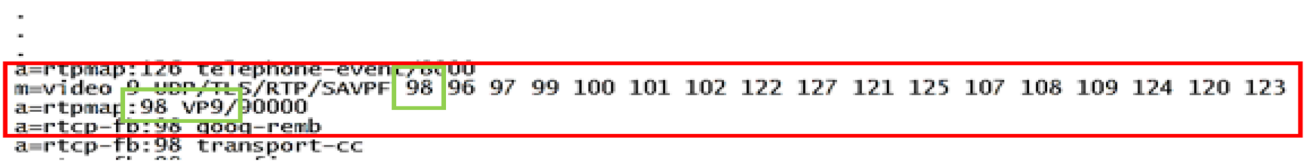
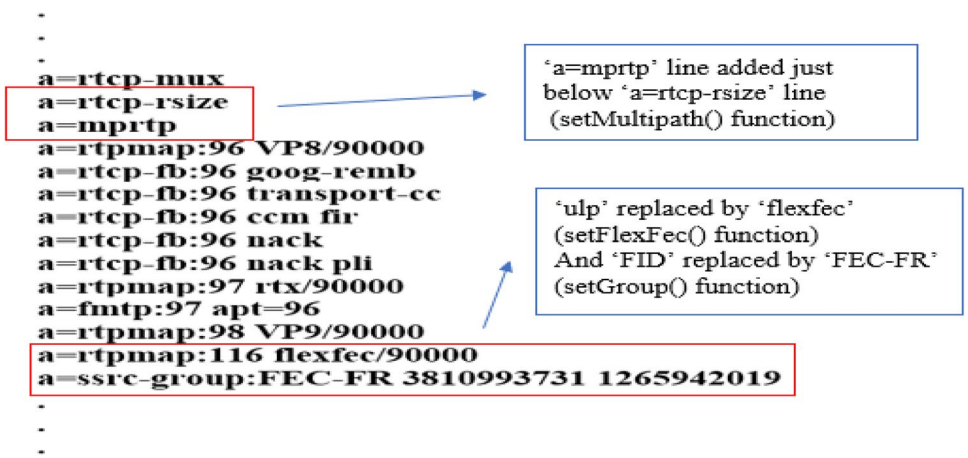



**Fig. 12** VP9 in answer

**Fig. 13** Extract of (SDP) answer after modification



of translation models such as the Translation API Basic, Translation API Advanced, and the AutoML Translation. In this project, the Translation API Basic model which uses Google's pre-defined neural machine translation to perform the translation [31] has been used. That is, the text is compared to an already defined database implemented by Google Cloud platform. Instant translations of texts into over 100 languages for websites and apps are possible with the basic version of the Translation API. Figure 14 shows how the application uses the API key to communicate with Google Cloud Platform, perform the translation using the cloud's services and then, sending the translated text back to the user.

The Google Cloud platform has been used to perform the translation which requires an API key to be included in the 'Translator.js' script for the application to be able to contact the cloud to perform the translation and subsequently retrieve the translated message from the cloud. An API key can be generated by creating a new project and adding 'Google Translation API' to it from the following link: https://console.cloud.google.com/home/dashboard

In the last step, an API key will be generated as shown in Fig. 15.

After generating the API key, it is added to the Translator.js script as shown in Fig. 16.

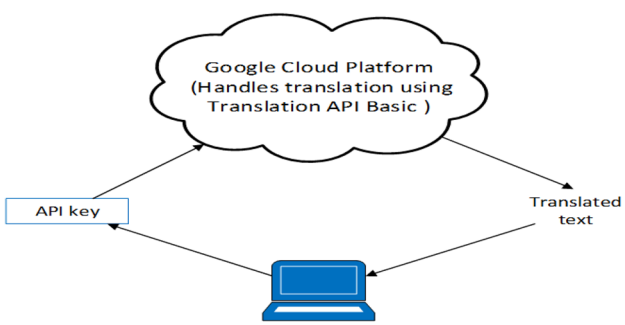



**Fig. 14** Communicating with the Google Cloud platform

```
var API_KEY = 'AIzaSyDag-1T8-EoN_hJSU8s2J0Gn6qq9A0hyRk';
```

**Fig. 16** Adding API key to Translator.js

The below steps show how the 'Translator.js' script uses the API key to perform Speech-to-Text conversion and communicate to Google Cloud platform.

1. Use SpeechRecognition API to perform Speech-to-Text conversion.
2. If user's microphone is working
   a. Contact Google Cloud Platform via the API key
3. else
   a. Alert "Failed capturing audio, i.e. microphone. Please check console-logs for hints to fix this issue."

As denoted by the steps above, the application uses the SpeechRecognition API to convert the input speech of a user to text (speech-to-text). The application then checks if the microphone of the user is working correctly and if everything is working fine, upon converting the speech into text, contact is made to Google Cloud Platform via the API key shown in Fig. 16. The idea here is to convert the speech into text and then translate the text. Therefore, if a user wants to translate a chat message directly in the chat box, only the speech-to-text conversion step will not be performed as the application is already dealing with text.

Using the GetVoices() function, the application performs a text-to-speech output by converting orthographic representation to speech. The goal of this function is to speak out the translated text displayed in the chat box. It provides the user a choice from a list of voices which can read in different languages such as English, French, Spanish and more. The GetVoices() function performs the following steps:

1. Populate list with voices

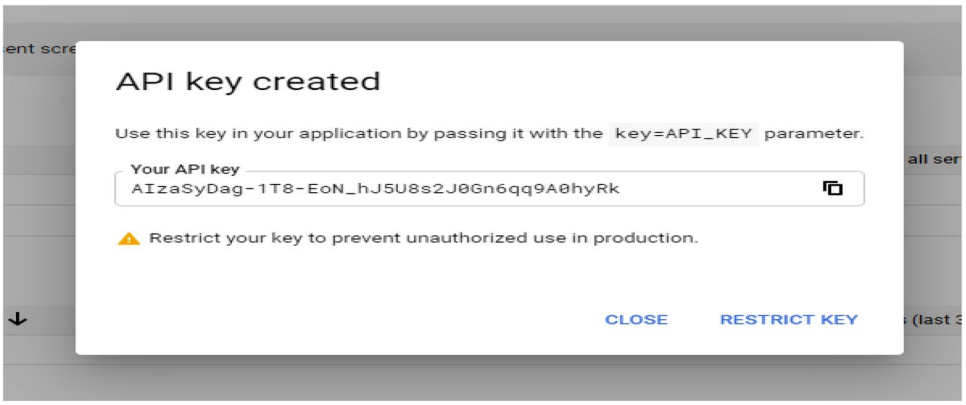



**Fig. 15** API key successfully generated

2. Speak recently displayed translated message from chat box

## SIP Client Server Implementation

A simplified overview of the functions implemented in the SIP based communication system platform is shown in Fig. 17. Figure 17 shows the SIP based communication platform being implemented and developed. The system consists of two sections namely the client side and the server side. On the client side, two applications have been developed. The first one is a Desktop App written in C# with JavaScript functionalities to handle SIP audio calls and SIP video calls. The second one is a mobile Android Application built in Java language to handle SIP audio calls. On the server side, two servers particularly Node.js for managing WebRTC peer-to-peer video calls and a FreeSwitch server (SIP server) for SIP audio and video calls have been set up.

### SIP Desktop App Implementation

For the implementation of the SIP VoIP Desktop Application in C#, the Ozeki VoIP SIP SDK which supports a first-rate technology for softphone development has been used. It is a software development kit for allowing VoIP calls using the SIP protocol. Ozeki VoIP SIP SDK is built on IETF standards (SIP, RTP, STUN, TURN, ICE, etc.) [32].

Figure 18 depicts the layout of the Desktop App designed using the drag and drop method on Visual Studio. It is the window that pops up when the Desktop App is launched. The application has been built to support the SIP protocol. It is a VoIP solution that enables audio and video calls via session initiation protocol. The SIP process is being handled by the software PBX of the FreeSwitch server (SIP server).

Table 2 describes the different functions of the desktop app interface.

### SIP Client Mobile App Implementation

Android has with an API that enables the incorporation of telephony functionalities of SIP in android applications. It provides a full SIP protocol stack and unified call management services that allow applications to set up voice calls.

To use the SIP API, the mobile application must create a SipManager object which is responsible for the following:

- establishing SIP sessions;
- establishing and receiving calls;
- verifying session connectivity;
- registering and unregistering with a SIP provider.

Every SIP account in an android SIP program is defined by a SipProfile. A SipProfile contains a SIP username, and information about the domain and application. The profile on the system running the application which is connected with the SIP account is called the local profile. The peer profile is the profile to which the session is connected to. A SIP client interfaces with the local SipProfile to the SIP server for sending SIP calls to a SIP address [33].

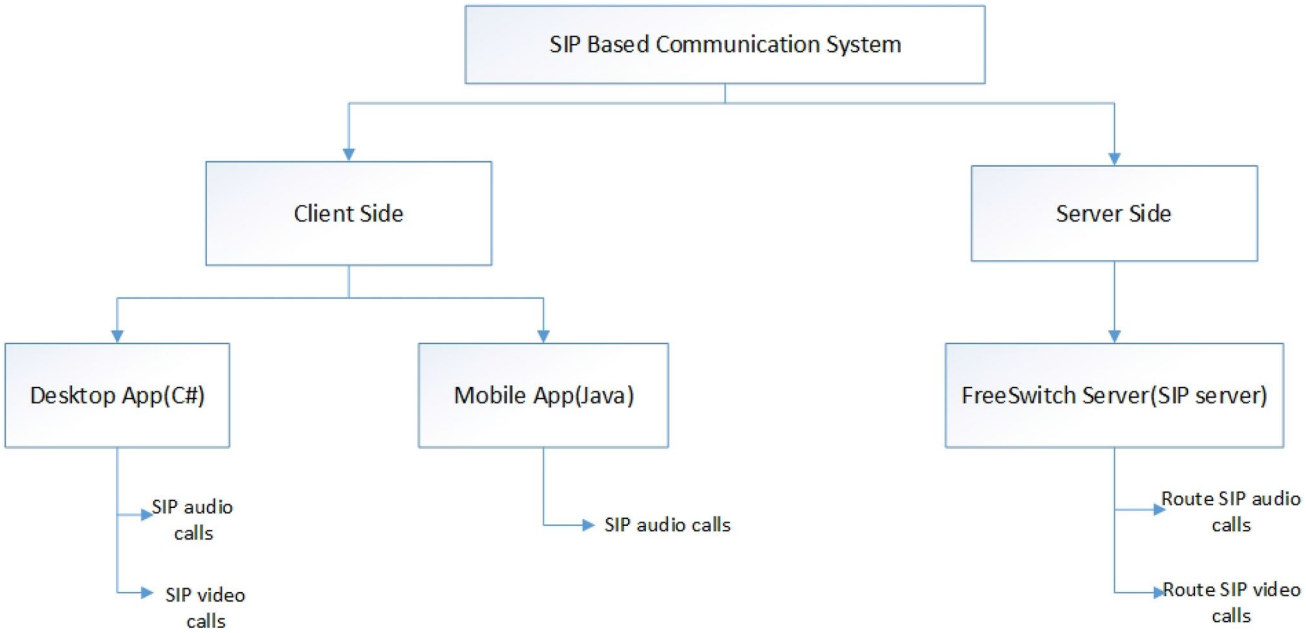



**Fig. 17** Overall system model

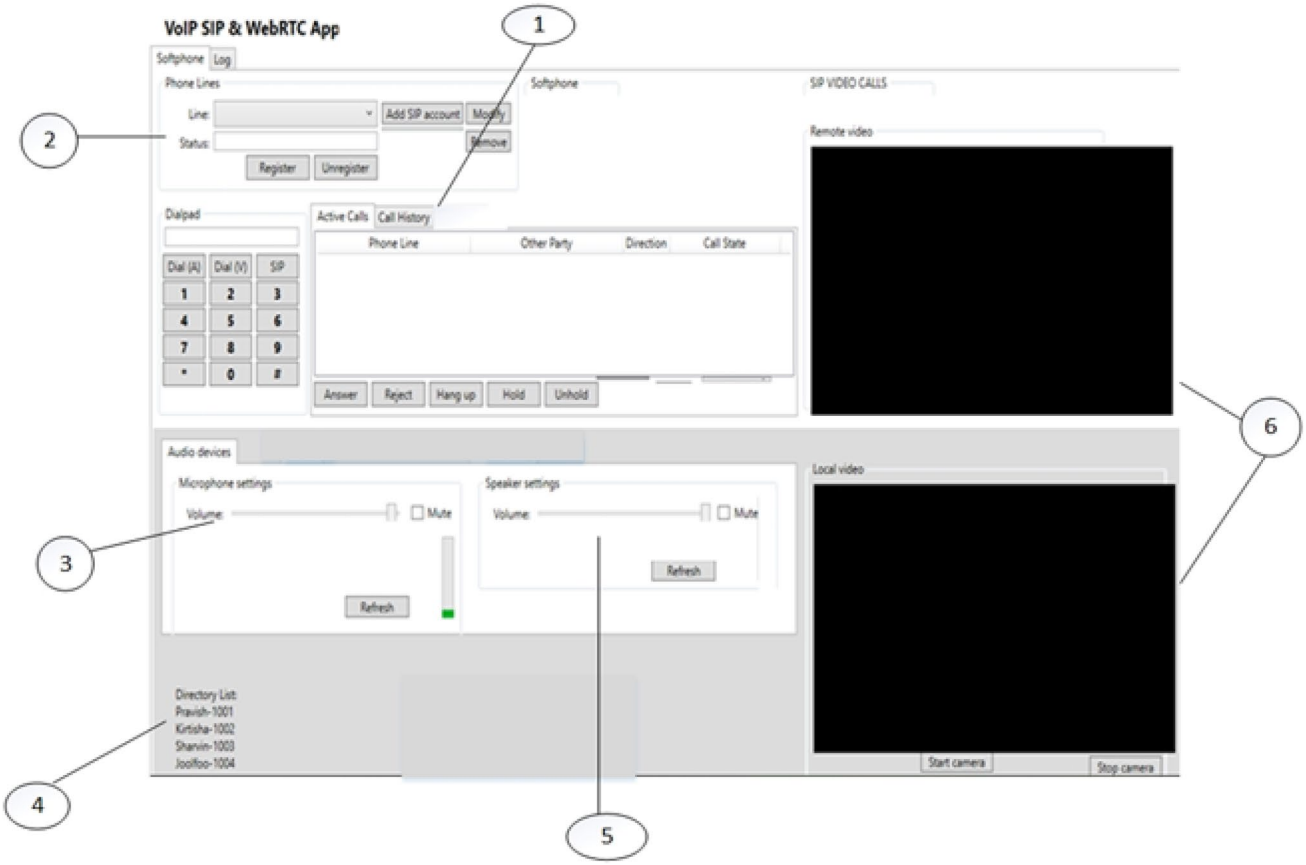



**Fig. 18** Desktop app interface

**Table 2** Description of the functions of the desktop application

| Task | Description |
|---|---|
| 1 | Displaying details about the ongoing call, call history and buttons for answering, rejecting, hanging up, holding and unholding |
| 2 | Registering the account of the user by specifying the username, password of the freeswitch to successfully be recognized by the server |
| 3 | Section to adjust volume of microphone, mute, refresh and a progress bar |
| 4 | Directory list showing the extensions of the contacts available to cal |
| 5 | Section to adjust the volume of speaker, mute and refresh |
| 6 | Remote and local video viewer used for video calls |

Figure 19 depicts the layout of the mobile application designed using the drag and drop method on Android Studio. It is the screen that pops up on mobile phones when the Android Icon is being clicked. The application has been built to support SIP protocol. It is a VoIP solution that enables audio calls via session initiation protocol. The SIP process is handled by the software PBX that is the FreeSwitch server (SIP server). The functions of the App are described in Table 3.

## SIP Server Implementation

The softphone is a VoIP solution that can be used to communicate audio or video via SIP. This is achieved by means of a PBX that manages the SIP accounts for the calls. The PBX is a software or a hardware that is basically the SIP server and establishes connection between the softphones. In this work, a software PBX called FreeSwitch has been programmed, configured and set up to route SIP audio and

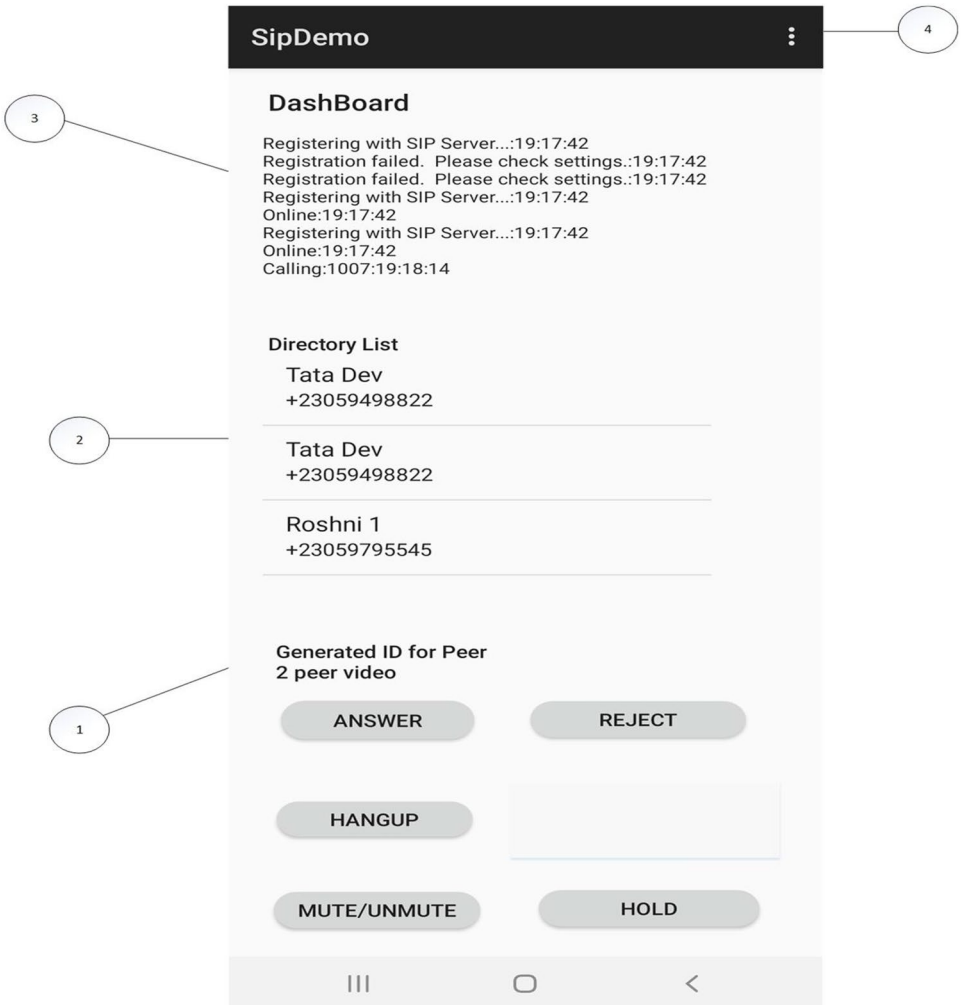



**Fig. 19** Interface of the android app

**Table 3** Functions of the mobile application

| Task | Description |
|---|---|
| 1 | TextView showing Generated ID to enter for peer-to-peer video call via WebRTC |
| 2 | Directory List to look for contact lists |
| 3 | Dashboard illustrating logs, call history and fired events |
| 4 | Pop up menu to call someone, edit SIP info and video conferencing |

video calls. FreeSWITCH is an open source communications platform. It is a soft switch which is able to handle thousands of simultaneous calls and that integrates different technologies like SIP (using the Nokia Sofia library), H.323, XMPP/Jingle, among others [34]. The server implementation requires some important directories and files. The most important configuration directories and files that will be used are described as follows:

Autoload_configs: It includes all configuration files and device modules on the platform.

Dialplan: It is the section where manipulation of Dial-digit is done. FreeSWITCH has a strong dialplan scheme that uses various filtering solutions based on different SIP headers for advance call routing. Perl regular expressions are utilized for dial-number handling of callers and callees. There are vital xml files located under/conf/dialplan directory which is default.xml and public.xml. Default.xml includes all of the dialed machine code point analysis. The signaling routes are evaluated here and it is flagged as the default dial plan configuration for all local users.

Public.xml is the xml file which is checked first for the incoming calls from external networks. Then the analysis takes place here according to match conditions.

Directory: This folder contains all user accounts, authentication keys and specific user features like Caller ID, among others. All local phone extensions are specified by default under/conf/directory/default. A separate XML file is created for each SIP extension which contains the user's call id, among others. The default.xml file is located under/conf/directory that contains all the user groups in the network.

Another important configuration file is the vars.xml file which is found under/conf directory and stores the system's globally used variables. It defines the SIP ports, the codec list, IP address for SIP signaling, among others [35].

Sip_profiles: It contains all necessary SIP configurations. Different profiles may be created. The Internal.xml file is found under conf/sip_profiles/directory and is used to monitor local extensions behavior. Different SIP settings and custom SIP options parameterization are added in this file. The external.xml file is also found under the conf/sip_profiles directory. It refers to "external" access which means connecting with VoIP providers.

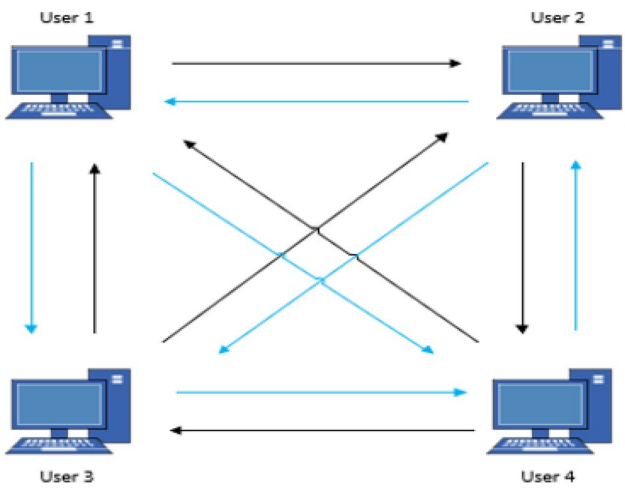



**Fig. 20** Video conferencing experimental setup

## System Testing and Performance Analysis

In this section, the experimental set-up for both the WebRTC and SIP systems are provided. The performance of the WebRTC system for video conferencing with four clients is first analysed with real-time translation and FEC optimization. After that a comparative analysis of peer-to-peer audio and video communication between WebRTC and SIP is performed.

### Testing the WebRTC Video Conferencing System with FEC Optimization

The video conferencing part was assessed with and without the optimization functions with up to four users, for both the VP8 and VP9 codec. The variation of video quality with reference to the number clients was analyzed by plotting peak signal-to-noise ratio (PSNR).

Figure 20 shows four users connected together in a mesh topology and this model has been used to perform the video conferencing experimentation.

All PCs that were used during testing were connected in the same LAN to a router of 10Mbps using Ethernet cable connection.

When the application is run by the initiator and four users enter the room (including the initiator), the window shown in Fig. 21 appears.

For the video conferencing part, videos of three minutes duration were recorded for all users in the room. After

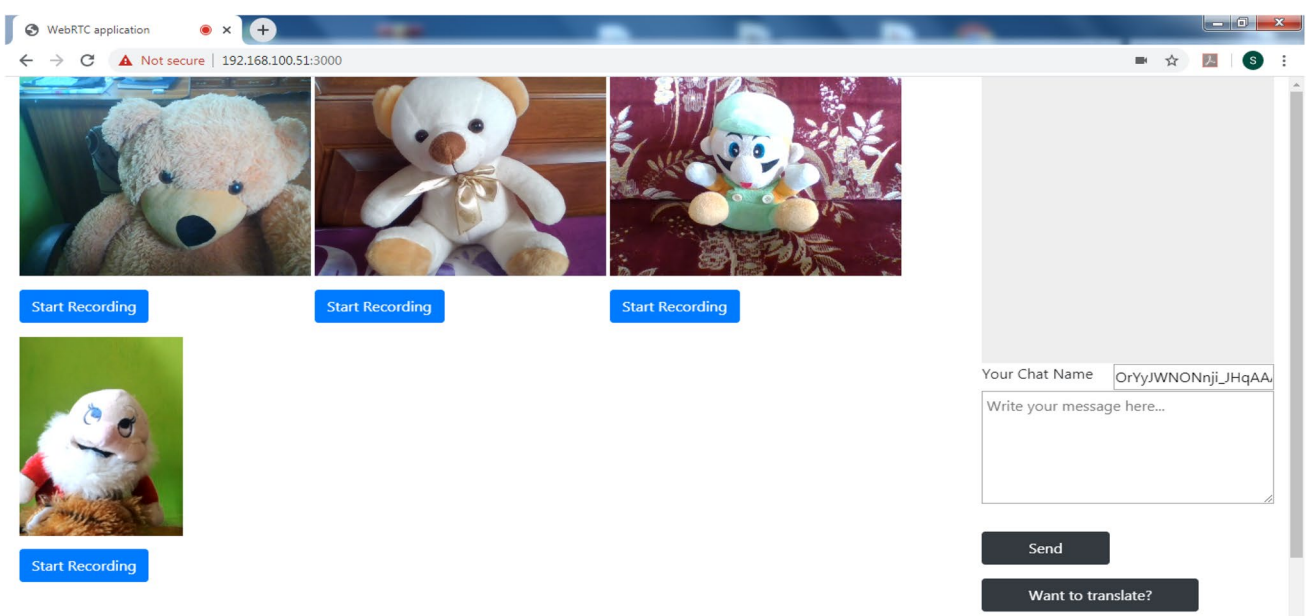



**Fig. 21** Video conferencing up to four users (initiator's screen)

that, each client's loca.l video was compared to their remote video that was obtained by other users.

The average PSNR values of all users were determined while using the VP8 codec. The trend of video quality, without optimization and with optimization, as the number of users increased is illustrated in Fig. 22.

Figure 22 shows an overall increase in the video quality each time a user joins in the room for VP8 codec after adding the SDP optimizations. The PSNR values increased to 36.5449 dB from 36.1335 dB with two users, to 30.3023 dB from 29.9844 dB with three users and to 25.8189 dB from 25.3839 dB with four users, all after applying the SDP optimizations.

The average PSNR values of all users were determined while using the VP9 codec. The trend of video quality, without optimization and with optimization, as the number of users increased is illustrated in Fig. 23.

Figure 23 shows an overall increase in the video quality after adding the SDP optimizations. The PSNR values increased to 36.4859 dB from 35.8429 dB with two users, to 30.9229 dB from 30.3334 dB with three users and to 26.5226 dB from 25.9909 dB with four users, all after applying the SDP optimizations.

Figure 24 shows the average PSNR values for both VP8 and VP9 up to four users for both un-optimized and optimized videos.

As can be seen from the figure, after making changes to the SDP for the VP8 codec, an average gain of 0.4114 dB and 0.4350 dB was obtained with two users and fours users respectively compared to unoptimized VP8. For the VP9 codec, after making changes to the SDP, an average gain of 0.6430 dB and 0.5324 dB was obtained with two users and three users respectively compared to unoptimized VP9. Comparatively, the average PSNR value of optimized VP8 is only slightly higher than optimized VP9, only by 0.059 dB, with two users. However, when considering cases with three and four users, optimized VP9 is higher than optimized VP8 by a margin of 0.6206 dB and 0.7037 dB respectively. The fact that VP9 is better than VP8 when it comes to resultant

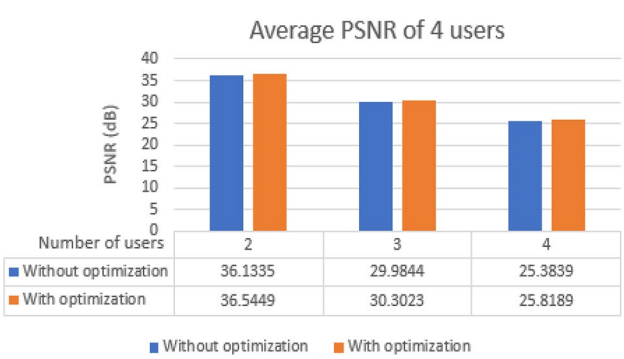



**Fig. 22** Comparing the average video quality (VP8) of four users

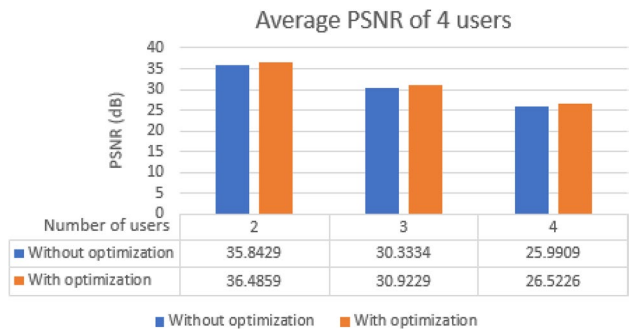



**Fig. 23** Comparing the average video quality (VP9) of four users

quality of compressed video, it was expected for optimized VP9 to show a better video quality than optimized VP8. It can also be deducted from the figure that both unoptimized VP8 and optimized VP8 perform better when dealing with only two users. On the contrary, unoptimized VP9 and optimized VP9 perform better when dealing with more than two users, that is, with three and four users.

## Testing the WebRTC System with Real-time Translation

Consider a conference room with four users, namely Evan, Fabrice, Sofia and David and the language they have chosen is as follows:

1. Evan: English
2. Fabrice: French
3. Sofia: English
4. David: English

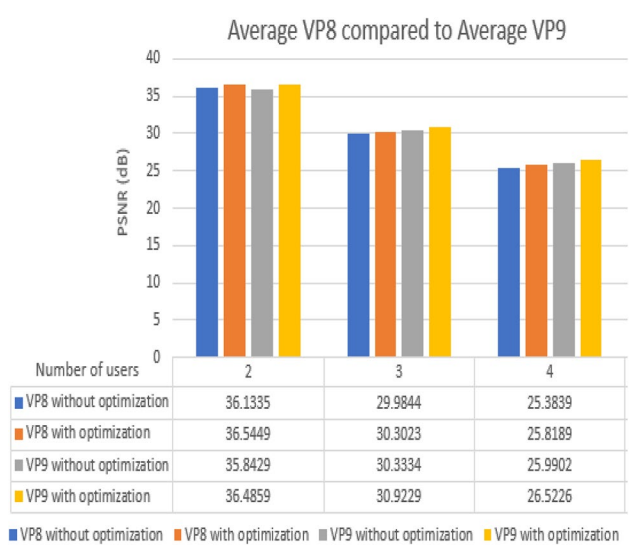



**Fig. 24** Comparing quality for average VP8 against average VP9

In this room up Evan is the initiator and has set the language as English and the three other users have set their language as French in the translation section.

Figure 25 shows the chat boxes of all users in the room, whereby the user Fabrice sent a message to every user in the room. The message 'Bonjour mes amis, comment allez-vous?' sent by Fabrice has been translated to the language specified at each user. Therefore, as Evan specified 'en (English)' on the application, the message was translated to 'Hello my friends, how are you?'. Eventually, the message has also been translated on Sofia's and David' applications as well.

To perform voice translation, the user has to use the 'Convert Your Voice' button as shown in Fig. 26.

On clicking the button, the application will perform the following steps:

1. Convert his speech into text (speech-to-text) using SpeechRecognition
2. Send the text to the other peers in the conference room
3. Translate the text depending on what the peers have chosen as their language
4. Update the chat box
5. Read recent text from chat box using selected voice from list (Text-To-Speech) using SpeechSynthesis

The aim here is to broadcast different lengths of speech in English from the initiator to every user in the room and note the translation delay in seconds at each user. Speech duration ranging from 5 s up to 25 s were used. The speech was generated using a dedicated text-to-speech software.

After the speech was broadcast to the other three users, the duration of the translated speech was measured as each client using the window setTimeout(function, milliseconds) method. The duration of the translated speech at each receiver was then compared to the duration of the speech sent by the initiator and the difference in time between the sent and received speeches were tabulated as the translation delay.

Table 4 shows the translation delay when they room consisted of fours users, that is, User 1 (initiator), User 2, User 3 and User 4. As observed in Table 4, the same lengths of speech have been sent using no translation (N/T) and with translation (W/T). As the duration of the speech was increased, the translation delay also increased ranging from 3.7 to 4.7 s. The translation delays started to become more or less constant as the duration of the speeches increased since after making first contact with the Google Cloud platform via the API key, a 'well-established link' between the application and the cloud was set up, allowing for translation to happen faster.

It can be deducted that as the number of users increased in the room, the translation delay increased as well. This may have resulted due to an increase in the amount of bandwidth that was currently being used or due to the increase in the number of requests made to the cloud platform to perform translation.

The trend of the delays, without translation and with translation with an increase in the duration of the speech is illustrated in Fig. 27.

## Comparing WebRTC with SIP for Peer-to-Peer Audio and Video Communication

The aim here is to analyze the variation of video quality and audio quality with reference to the number of clients by plotting peak signal–noise ratio (PSNR) against number of client pairs and also by plotting segmental signal-to-noise ratio (SSNR) against number of client pairs for making the contrast of audio quality. The following are tests performed to evaluate the quality of audio and video.

- Desktop App-Desktop App (PSNR for evaluating video quality using SIP AND WEBRTC).
- Desktop App-Desktop App (SSNR for evaluating audio quality using SIP AND WEBRTC).
- Desktop App-Desktop App (PSNR for evaluating video quality using SIP AND WEBRTC).

In all, five laptops were used whereby the client app was opened-up twice which makes a total of 5 pairs of clients to set up the testbed. The experiments and tests were carried out on the same LAN. During the tests, the router and the internet speed which was 10Mbps were kept constant. Figure 28 illustrates the experimental setup used for testing whereby each pair of clients established duplex transmission of data.

In Fig. 29, it can be clearly seen that the audio quality of WebRTC peer-to-peer is much lower than the SIP audio calls. Hence, from the results obtained, we can conclude that SIP protocol offers a better service in terms of audio quality. Ozeki VoIP SDK offers an excellent sound quality since it embeds advanced digital voice processing in its software development kit such as noise cancellation, adaptive jitter buffering and echo cancellation [36]. WebRTC uses Opus and G.711 as audio codec with no optimization in its browser stack [37]. Thus, this explains why the quality of SIP is better than WebRTC. The SSNR quality of both SIP and WebRTC both drops since both Node.js and FreeSwitch server need to handle more clients at the same time.

In terms of video quality, results in Fig. 30 demonstrate that for WebRTC, the PSNR for the first client pair is much higher than the SIP video quality for first client pair of SIP. However, as more simultaneous video streams are being established for both SIP and WebRTC, there is more significant drop for WebRTC than for SIP. Hence,

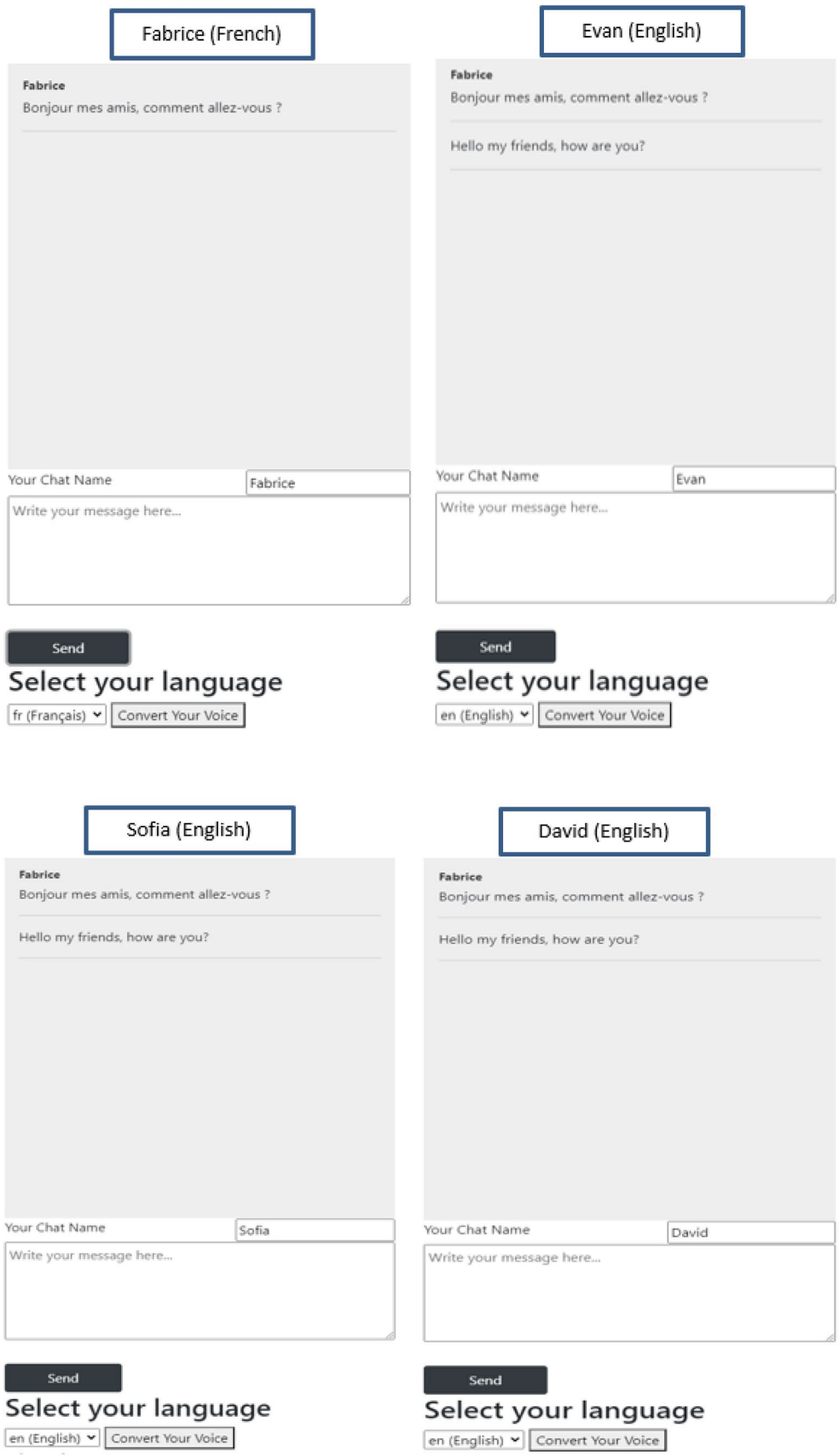



**Fig. 25** Translation up to four users

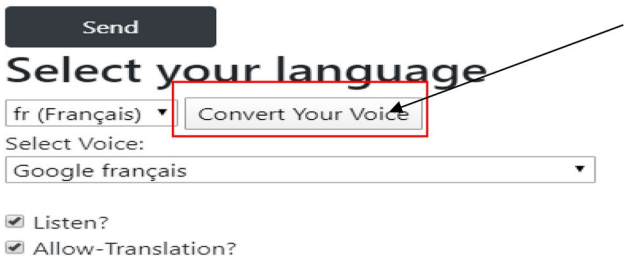



**Fig. 26** Translating voice

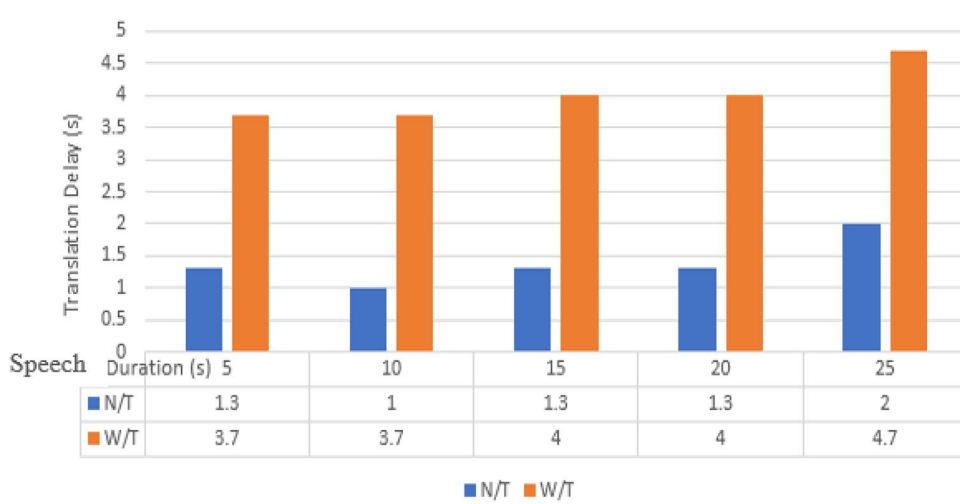
with more users using the system at the same time, SIP will be better to use than WebRTC. The drop in PSNR for SIP was 26.05–20.84 dB whereas for WebRTC, the PSNR fell from 39.16–20.84 dB. WebRTC uses VP8 as the primary video codec for real video transmission. VP8 uses 14 bits in terms of width and height which creates a 16,384 × 16,384 pixels of high resolution and does not have any limitation on the frame rate or data size. SIP uses H.246 as the video codec and is often compared with VP8 [38]. Several analysts also noticed that the two codecs are of good standard quite close. However, during the above comparison the video quality of WebRTC has been higher than SIP. Moreover, FreeSwitch has been found to handle more than 1000 of clients compared to Node.js which has a limitation around 15 clients simultaneously [39]. This relates why the drop in SIP video quality is lower compared to WebRTC.

As illustrated in Fig. 31, the SIP audio call quality established between the mobile applications is better than SIP audio calls established between the desktop applications which in turn is better than that set up between desktop app and mobile app which is cross-platform. The SIP audio call quality of mobile applications is better. So, the SIP android API seems to offer a more optimized SDK in terms of audio codecs than the desktop applications. The cross-platform audio calls are lower in quality since audio calls are being established between a mobile application written in Java and a desktop application written in C#.

**Table 4** Translation delay with four users

| Number of users: 4 | | | | | | |
|---|---|---|---|---|---|---|
| Speech duration | Delay | | | | | |
| | User 2 | | User 3 | | User 4 | |
| | N/T | W/T | N/T | W/T | N/T | W/T |
| 5 | 1 | 2 | 2 | 4 | 1 | 5 |
| 10 | 1 | 2 | 1 | 4 | 1 | 5 |
| 15 | 1 | 3 | 1 | 4 | 2 | 5 |
| 20 | 1 | 2 | 2 | 4 | 1 | 6 |
| 25 | 2 | 3 | 2 | 4 | 2 | 7 |
| Average speech duration (*N/T*) | 1.3 | 1 | 1.3 | 1.3 | 2 | |
| Average speech duration (*W/T*) | 3.7 | .37 | 4 | 4 | 4.7 | |

**Fig. 27** Comparing translation delay with four users



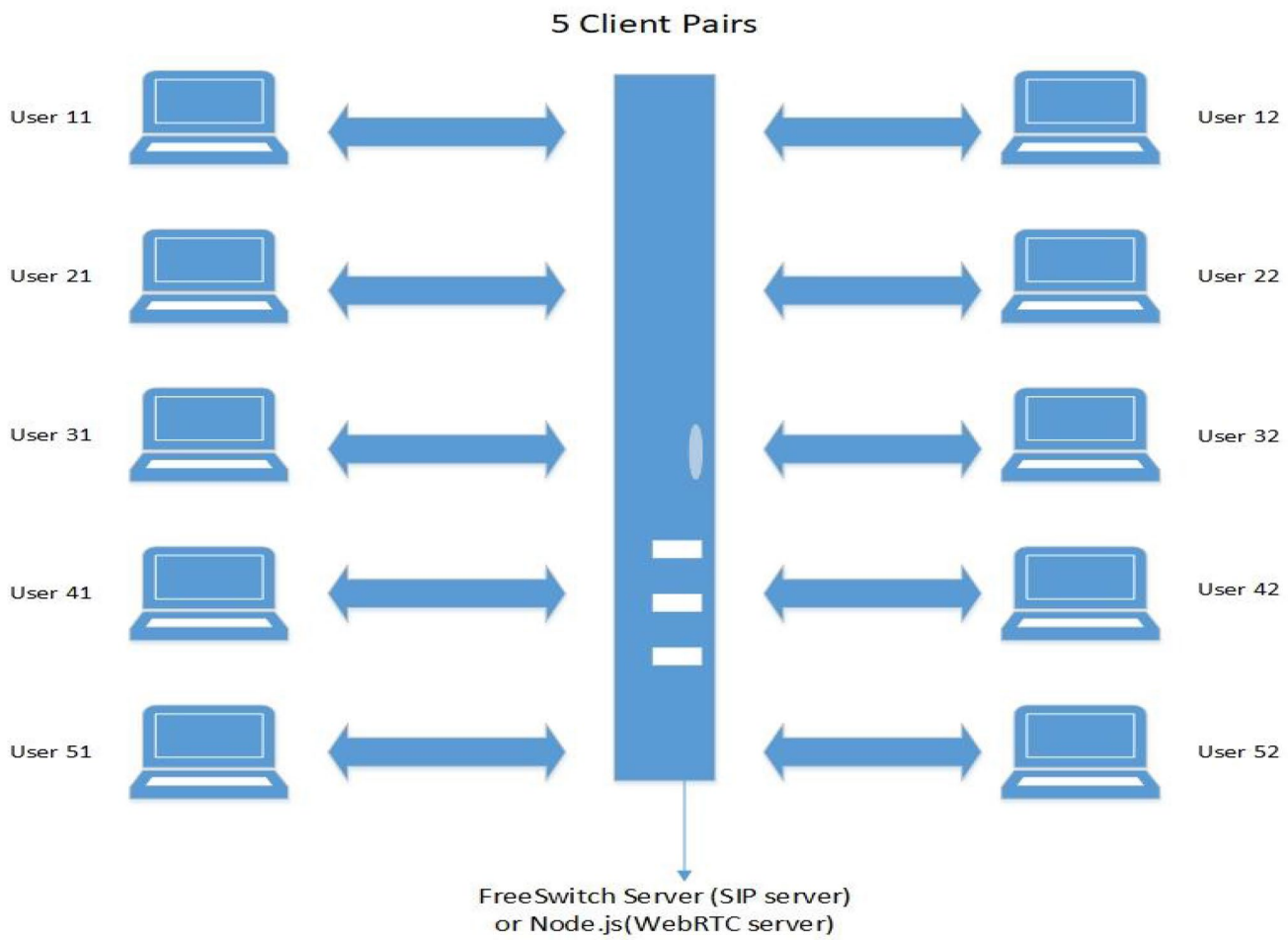



**Fig. 28** Experimental setup for testing

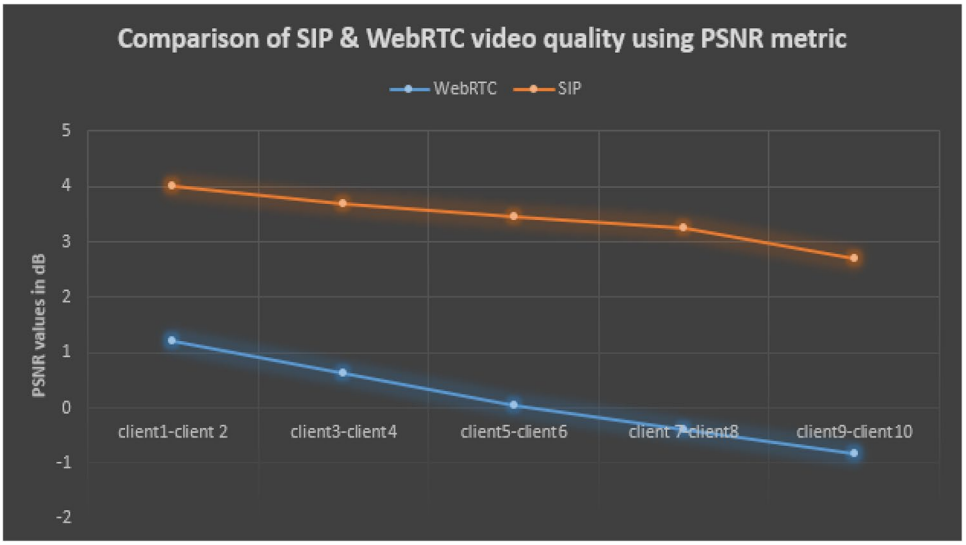



**Fig. 29** Comparison of SIP audio quality with WebRTC audio quality for desktop-to-desktop APP

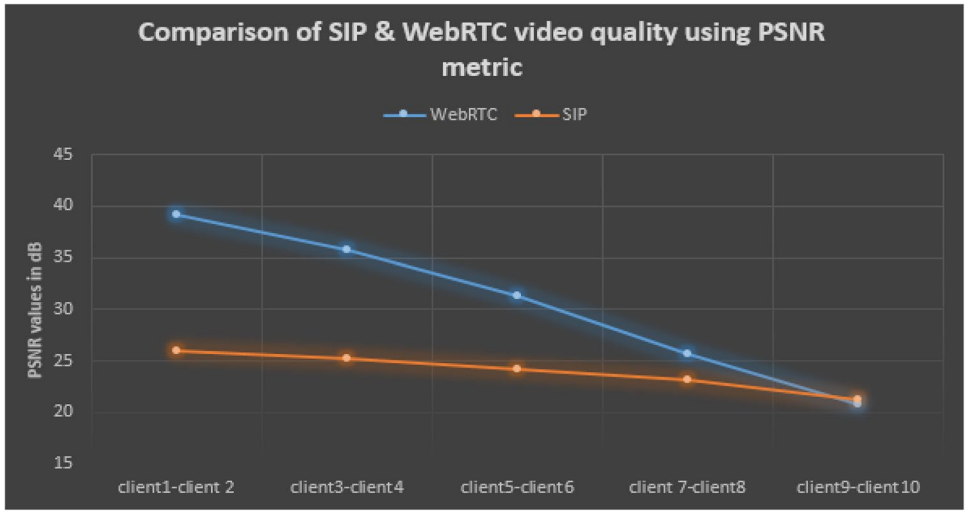



**Fig. 30** Comparison of video quality between SIP and WebRTC for desktop-to-desktop APP

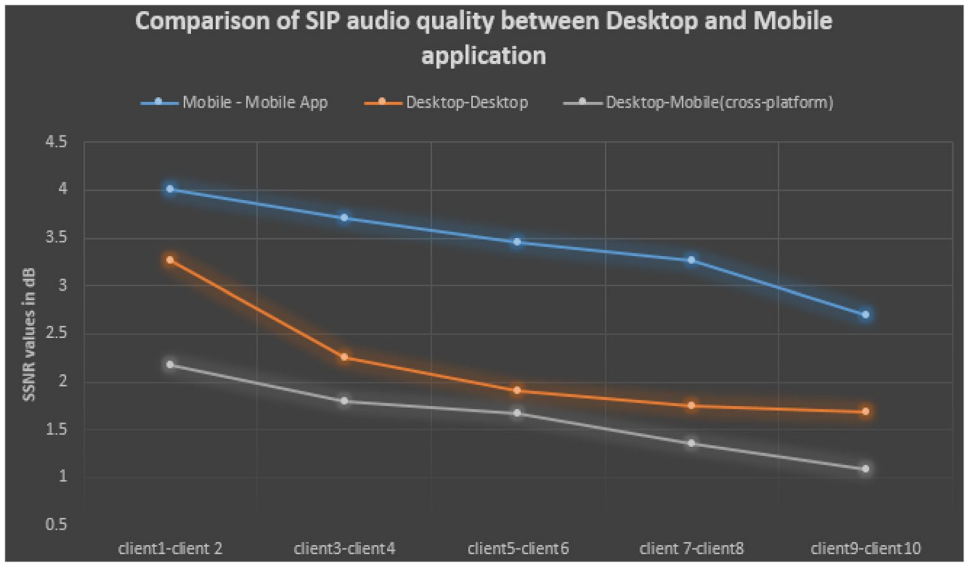



**Fig. 31** Comparison of SIP audio call between mobile and desktop app

## Conclusion

In this paper two real-time video and audio communications systems based on SIP and WebRTC respectively have been set-up and their performances analysed. It was observed that WebRTC peer-to-peer audio communication provides a much lower quality in terms of SSNR than the SIP audio calls. SIP audio calls provided an average gain of 3 dB in SSNR when five client pairs were used. For video communications, WebRTC provided a much better quality in terms of PSNR than SIP with a gain of up to 15 dB in PSNR when one client pair was used. However, this gain dropped to 2 dB with four client pairs and subsequently to 0 when there were five client pairs. It was also observed that SIP audio calls established between two mobile devices provided a significantly better SSNR than two desktop devices. Furthermore, the performance of the WebRTC application was evaluated with different codecs in terms of PSNR as well as the impact of the real-time speech translation delay. For the VP8 codec, after making changes to the SDP, an average gain of 0.4114 dB and 0.4350 dB was obtained with two users and fours users, respectively, compared to unoptimized VP8. For the VP9 codec, after making changes to the SDP, an average gain of 0.6430 dB and 0.5324 dB was obtained with two users and three users respectively compared to unoptimized VP9. When the speech translation delays for up to four users were measured and analyzed, translation delays ranging from 2 s (with two users) to 4.7 s (with four users) were recorded. It was noticed that as the number of users in the room increased, the translation delay also increased slightly. A possible future work will be to analyses the performance of SIP and WebRTC systems when more servers along with load balancing mechanisms are incorporated.

**Acknowledgements** The authors would like to thank the University of Mauritius for providing the necessary facilities to conduct this research.

## Compliance with Ethical Standards

**Conflict of interest** On behalf of all authors, the corresponding author states that there is no conflict of interest.

## References


1. Adobe: Creative, marketing and document management solutions. Adobe: Creative, marketing and document management solutions, 2020. https://www.adobe.com/. Accessed 17 Jan 2020.
2. Loreto S, Romano S. Real-time communication with WebRTC. 1st ed. Sebastopol: O'Reilly Media; 2014. p. 1–6, 52 ((**ISBN 978-1-449-37187-6**)).
3. VoIP Think—SIP Architecture: session initiation protocol—RFC 3261. https://www.voipthink.com/SIP/SIP_architecture.php. Accessed Mar 2020.
4. Aye TZ. Performance analysis between H.323 and SIP over VoIP. Int J Trend Sci Res Dev (IJTSRD). 2019. https://doi.org/10.31142/ijtsrd26647.
5. Namratha N et al. Survey on detection techniques for denial-of-service attacks in SIP-based voice over IP networks. In: Proceedings of second international conference on emerging trends in science & technologies for engineering systems (ICETSE-2019), Chickballapur, Karnataka, India, 2019. https://doi.org/10.2139/ssrn.3509078.
6. Jackulin C, et al. Framework of Sip-Based Centralized Multimedia Conferencing System. Int J Res Comput Appl Robot. 2015;3(2):50–8.
7. Jahangir A et al. Performance evolution of QoS in VoIP using SIP for ad hoc routing protocols. Int J Comput Sci Inf Secur (IJCSIS). 2017;15(3):332–337.
8. Zaker AM et al. Android based encrypted IP voice call communication through SIP server on 3G GPRS/Wi-Fi. Int J Comput Sci Inf Secur (IJCSIS). 2017;15(4)
9. Xavi D, et al. IP speaker with SIP and smart notification system. Int J Netw Syst. 2019. https://doi.org/10.30534/ijns/2019/09832019.
10. Edan N, Al-Sherbaz A, Turner S. Design and implement a hybrid WebRTC signalling mechanism for unidirectional & bi-directional video conferencing. Int J Electric Comput Eng IJECE. 2018;8(1):390. https://doi.org/10.11591/ijece.v8i1.pp390-399.
11. Vucic D, Skorin-Kapov L, Suznjevic M. The impact of bandwidth limitations and video resolution size on QoE for WebRTC-based mobile multi-party video conferencing. In: 5th ISCA/DEGA workshop on perceptual quality of systems (PQS 2016), Berlin/Germany, 2016. https://doi.org/10.21437/pqs.2016-13. Accessed 18 Jan 2020.
12. Maruschke M, Zimmermann J, Haensge K, Bach T. The role of QoS in WebRTC and IMS-based IPTV services. Int J Adv Telecommun. 2015;8(1 & 2):59–68. https://doi.org/10.13140/2.1.2224.8487.
13. Shiga Y, Takami K. Development and evaluation of a contact center application system to integrate multiple communication channels using WEBRTC. Int J Comput Netw Commun. 2017;9(3):1–20. https://doi.org/10.5121/ijcnc.2017.9301.
14. Shanmugapriya K, Shanthi BP. End to end security enhancement in SIP using Ssas for ad-hoc network. Int J Mobile Netw Commun Telemat. 2014;4(6):37–9. https://doi.org/10.5121/ijmnct.2014.4604.
15. Parmar N, Ranga V. Performance analysis of WebRTC and SIP for video conferencing. Int J Innov Technol Explor Eng. 2019;8(9):679–86. https://doi.org/10.35940/ijitee.i1109.0789s19.
16. Barik R, Welzl M, Elmokashfi A, Dreibholz T, Gjessing S. Can WebRTC QoS Work? A DSCP measurement study. In: 2018 30th international teletraffic congress (ITC 30), Vienna/Austria, 2018. https://doi.org/10.1109/itc30.2018.00034.
17. Rouhana N, Horlait E. Differentiated services and integrated services use of MPLS. In: Proceedings ISCC 2000 fifth IEEE symposium on computers and communications, Antibes/France, 2000. https://doi.org/10.1109/iscc.2000.860638.
18. Barik R, Welzl M, Elmokashfi AM, Gjessing S, Islam S. fling: a flexible ping for Middlebox measurements. In: Proceedings of the 29th international teletraffic congress (ITC), Genoa/Italy, 2017 **(ISBN 978-0-9883045-3-6)**
19. Carlucci G, De Cicco L, Holmer S, Mascolo S. Analysis and design of the google congestion control for web real-time communication (WebRTC). In: Proceedings of the 7th international conference on multimedia systems (MMSys), Klagenfurt/Austria, 2016. https://doi.org/10.1145/2910017.2910605.
20. Li W, Wang Z, Ho D, Wei G. On boundedness of error covariances for Kalman consensus filtering problems. IEEE Trans Autom Control. 2019. https://doi.org/10.1109/tac.2019.2942826.
21. Garcia B, Gortazar F, Lopez-Fernandez L, Gallego M, Paris M. WebRTC testing: challenges and practical solutions. IEEE Commun Stand Mag. 2017;1(2):36–42. https://doi.org/10.1109/mcomstd.2017.1700005.
22. Salini R, Safrin P, Shanmugapriyaa P, Sindhu S. Switching between multiple languages based on speech recognition and translation. Int J Eng Res Technol (IJERT). 2018. https://doi.org/10.17577/ijertcon033.
23. Emmanuel E, Dirting B. A peer-to-peer architecture for real-time communication using Webrtc. J Multidiscipl Eng Sci Stud (JMESS) 2017;3(4):1671–1683.
24. Introduction to the getUserMedia API. https://www.sitepoint.com/introduction-getusermedia-api/. Accessed Mar 2020.
25. MDN web. Mozilla and individual contributors. 2005. https://developer.mozilla.org/en-US/docs/Web/API/RTCPeerConnection. Accessed Mar 2020.
26. RTCPeerConnection. MDNweb. https://developer.mozilla.org/en-US/docs/Web/API/RTCPeerConnection. Accessed Mar 2020.
27. How Network Address Translation Works. Howstuffworks?. https://computer.howstuffworks.com/nat.htm. Accessed Mar 2020.
28. Dutton S. WebRTC in the real world: STUN, TURN and signaling. https://www.html5rocks.com/en/tutorials/webrtc/infrastructure/. Accessed Mar 2020.
29. MDN Web Docs. WebRTC connectivity. Available online: https://developer.mozilla.org/en-US/docs/Web/API/WebRTC_API/Connectivity. Accessed Mar 2020.
30. EFORT (2020). SIP: session initiation protocol. Available Online: https://www.efort.com/media_pdf/SIP_EFORT_ENG.pdf. Accessed 10 July 2020.
31. Google. Google Cloud Platform | Documentation, Google Cloud, 2020. Available Online: https://cloud.google.com/docs. Accessed 25 Feb 2020.
32. Ozeki Ltd. Introduction to Ozeki VoIP SIP SDK. Available Online: https://www.voip-sip-sdk.com/p_7273-voip-introduction-to-voip.html. Accessed Mar 2020.
33. developers android. Session initiation protocol overview. Available Online: https://developer.android.com/guide/topics/connectivity/sip. Accessed Feb 2020.
34. VoIP-Info.org. Freeswitch - What it is. Available Online: https://www.voip-info.org/freeswitch/. Accessed Mar 2020.
35. Stelios Antoniou. How to Install FreeSWITCH in Windows. Available Online: https://www.pluralsight.com/blog/it-ops/how-to-install-freeswitch-windows. Accessed Mar 2020.

36. Ozeki Ltd. Voip voice quality. Available Online: 2020. https://www.voip-sip-sdk.com/p_7309-voip-voice-quality-voip.html. Accessed 22 June 2020.
37. MDN Web Docs. Codecs used by WebRTC. Available Online: https://developer.mozilla.org/en-US/docs/Web/Media/Formats/WebRTC_codecs. Accessed 22 June 2020.
38. Onsip. VP8: The WebRTC Video Codec. Available Online: https://www.onsip.com/voip-resources/voip-fundamentals/webrtc-video-codec. Accessed 22 June 2020.
39. FreeSWITCH. FreeSWITCH Explained. Available Online: https://freeswitch.org/confluence/display/FREESWITCH/Specifications. Accessed 22 June 2020.

**Publisher's Note** Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.